

RA3- SEÑALES, COMPONENTES REUTILIZABLES

Instrucciones

Objetivo: integrar un componente reutilizable en MiniOffice y documentar cómo se comunica con la aplicación mediante señales.

Parte del archivo adjunto **contadorWidget.py**, que contiene la clase `WordCounterWidget`. El componente muestra palabras, caracteres y una estimación del tiempo de lectura.

1. Integra el componente

1. Incorpora el módulo a tu proyecto e importa su clase.
 2. Crea una instancia del contador y colócala en una zona visible de la interfaz, por ejemplo la barra de estado.
 3. Sustituye el contador anterior de palabras y caracteres por este componente. El cálculo debe quedar en el componente, sin mantener otra implementación paralela en la ventana.
-

2. Conecta el editor y las señales

Cada vez que cambie el documento, pasa su texto actual a `update_from_text(text)`. Ten en cuenta que `QTextEdit.textChanged` avisa del cambio, pero no envía el texto como argumento: la conexión debe obtenerlo del editor.

El componente emite `conteoActualizado(int, int)` después de actualizarse. Sus dos argumentos son, en este orden, **palabras** y **caracteres**. Conecta un receptor que permita comprobar que la ventana recibe estos datos.

El contador debe quedar sincronizado al iniciar, escribir, pegar, borrar, abrir un documento o crear uno nuevo. Evita duplicar conexiones cada vez que abres un archivo y evita que la recepción del conteo vuelva a modificar el texto.

3. Documenta el uso y las señales

Añade al README.md del proyecto:

- Qué hace el componente, dónde se encuentra y cómo se incorpora a otra ventana.
- Qué recibe `update_from_text` y cómo se obtiene el texto del editor.
- Una relación de las señales utilizadas: nombre, emisor, momento de emisión, argumentos y función receptora.
- La secuencia completa: cambia el editor → se actualiza el componente → se emite el conteo → lo recibe la ventana.
- Configuración disponible: `wpm`, `mostrarPalabras`, `mostrarCaracteres` y `mostrarTiempoLectura`, con una explicación breve de cada opción.

Explica las reglas del archivo proporcionado: los caracteres incluyen espacios y saltos de línea; las palabras se calculan con su expresión regular. El tiempo de lectura es una estimación a partir de `wpm`.

Incluye también las señales que use el resto de tu aplicación: acciones del menú, botones, buscador y controles de formato. Si has incorporado voz, documenta sus señales de texto reconocido, estado o error. Describe tus conexiones reales, con los nombres de tus métodos.

Comprueba el resultado y entrega

- Documento vacío: **0 palabras y 0 caracteres**.
- Texto `Hola mundo`, sin salto final: **2 palabras y 10 caracteres**.
- Texto `Hola, mundo!`, sin salto final: **2 palabras y 12 caracteres**.
- Comprueba también pegar, borrar, abrir y crear documentos. El dato mostrado y el recibido por la señal deben coincidir.

Entrega el proyecto con el componente integrado y el README actualizado. Incluye una captura del contador y una breve evidencia de que el receptor recibe ambos valores.

Consulta: [señal `textChanged` de `QTextEdit`](#).

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



contadorWidget.py

PY · 1,6 KB

